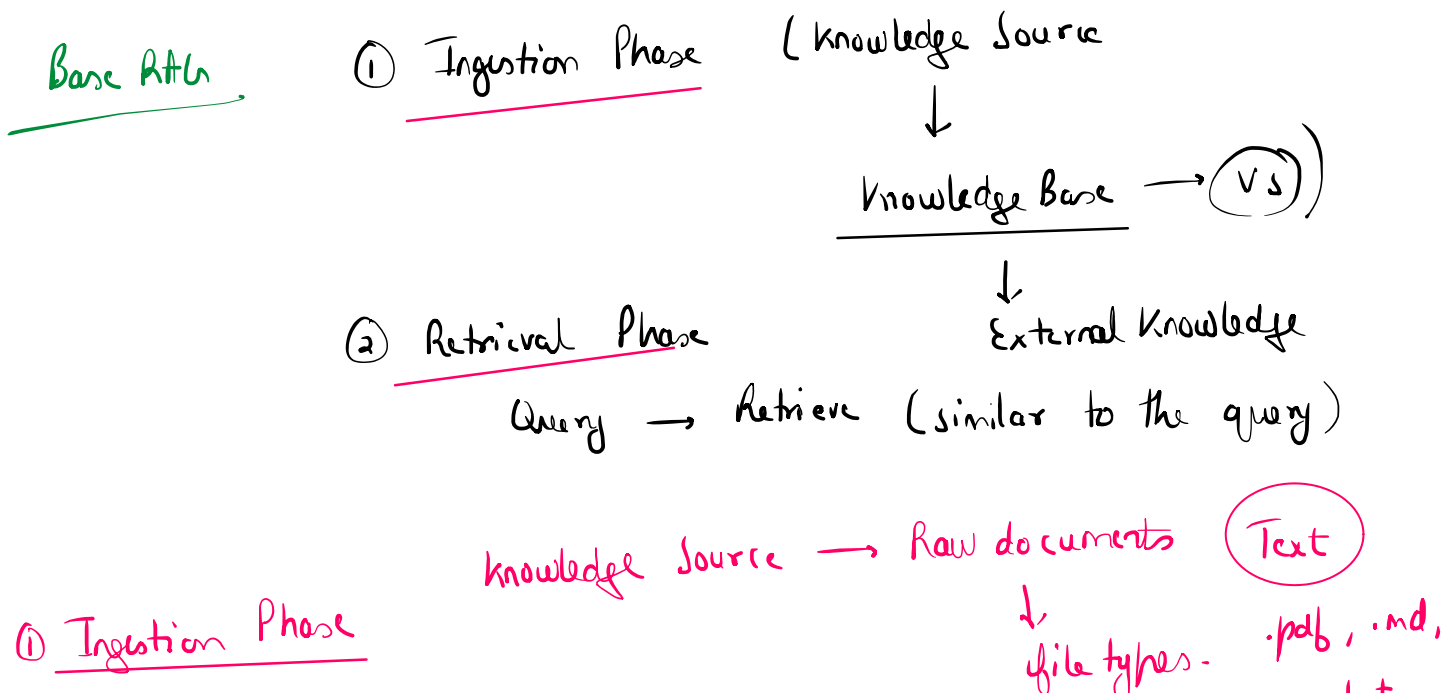
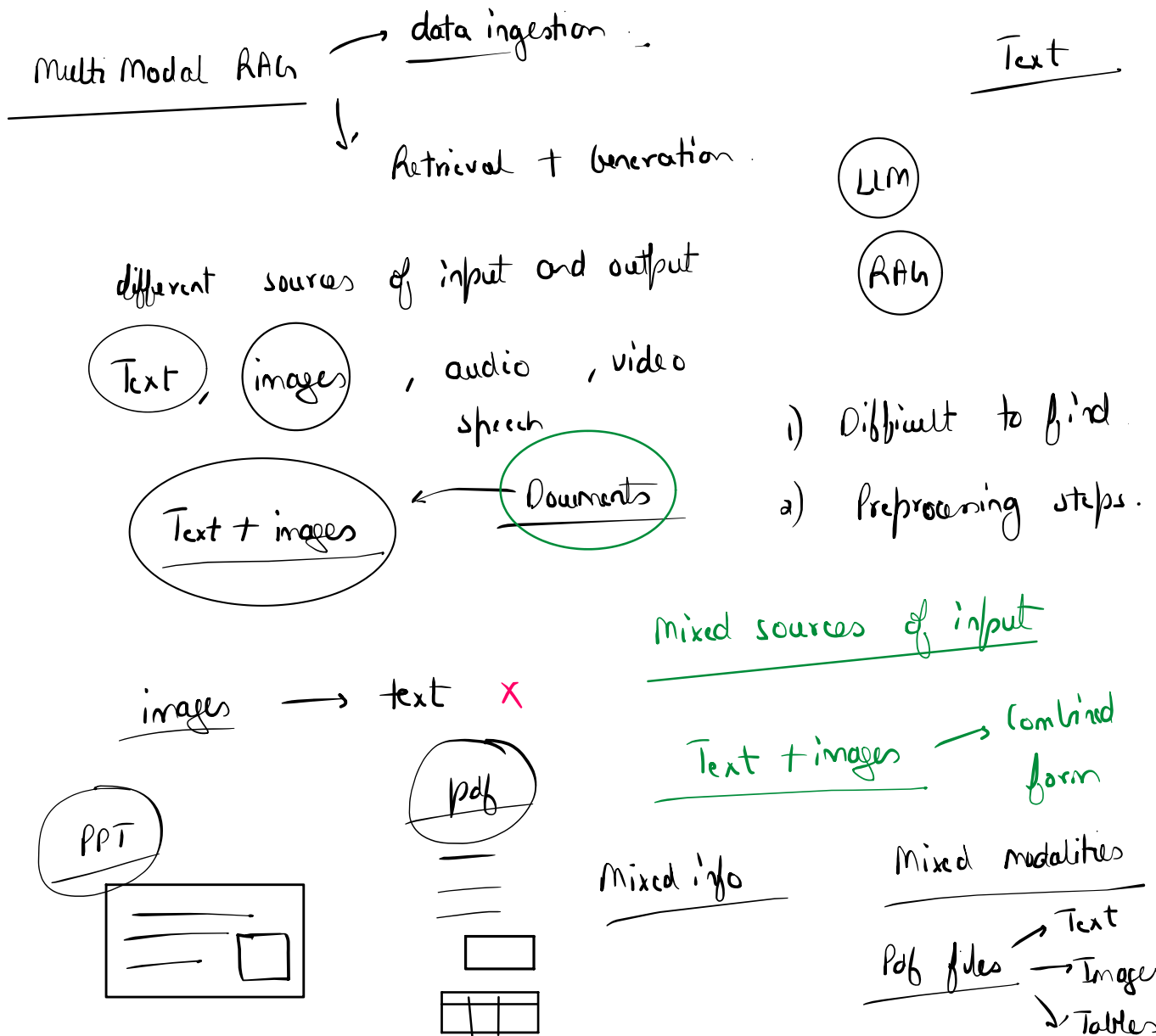




① Ingestion Phase

① Document loaders → Load, Parse, Extract
↓
Unified format

↓
file types - .pdf, .md, .txt, .py
Text ←

② Splitting

Text
encode.
↓
Chunks

③ Embedding vectors → Numerical Representation

Semantic meaning → Dense Representation
↓
Dense Vectors

dimensionality

features → Values
↓
metrics / patterns

Doc / Text Embeddings -

semantic meaning

④ Vector Stores → Database

Store the embeddings } Similarity
Index them } search

Text Query → Query vector → (VS) → Similarity search

Input Prompt

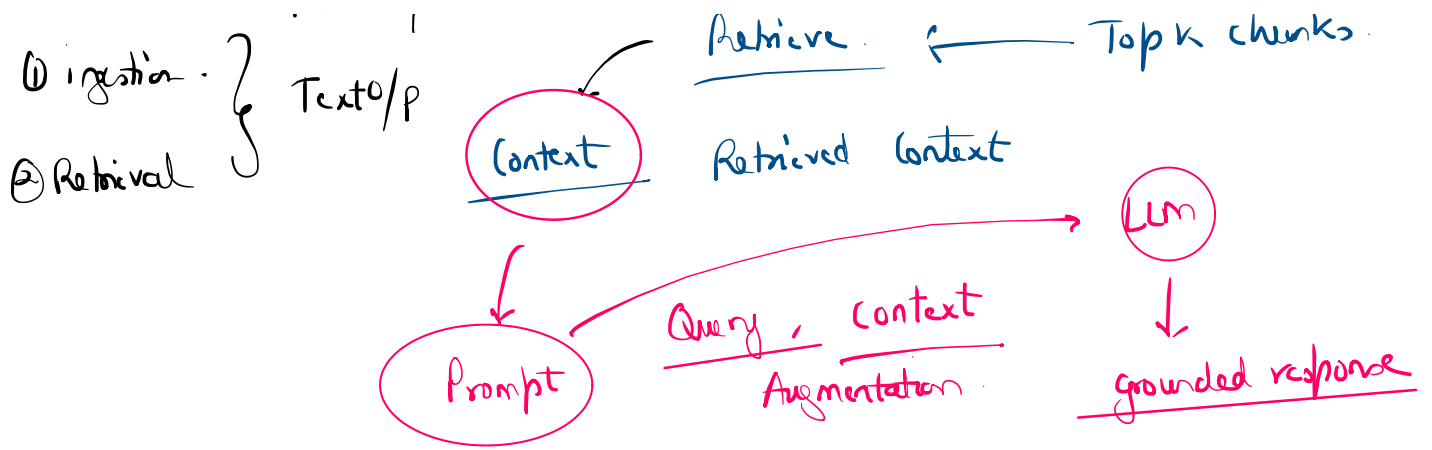
Embedding vectors → Semantic similarity

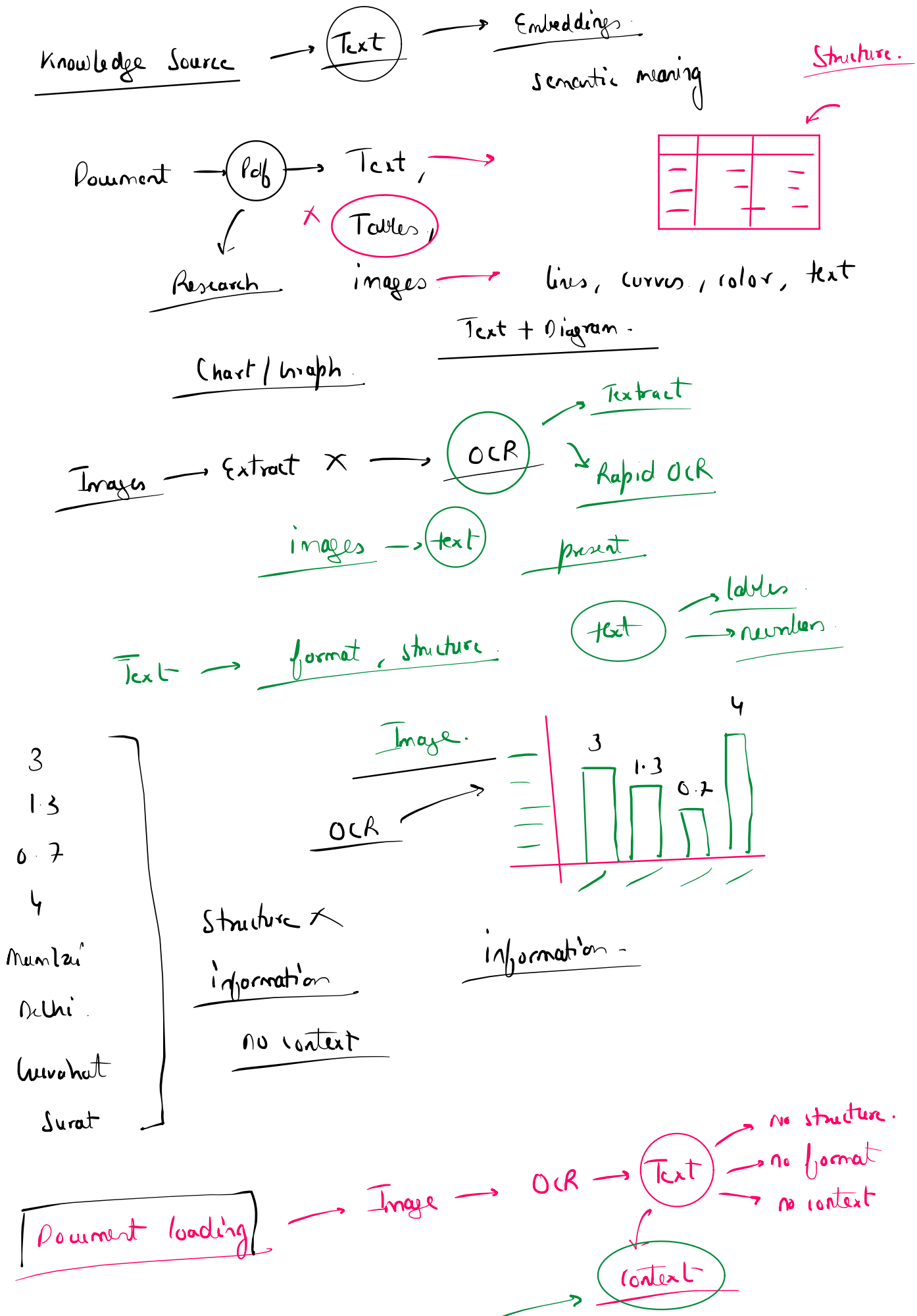
⚡ (Text)

↓

① ingestion - ?
Text I/P
Text O/P

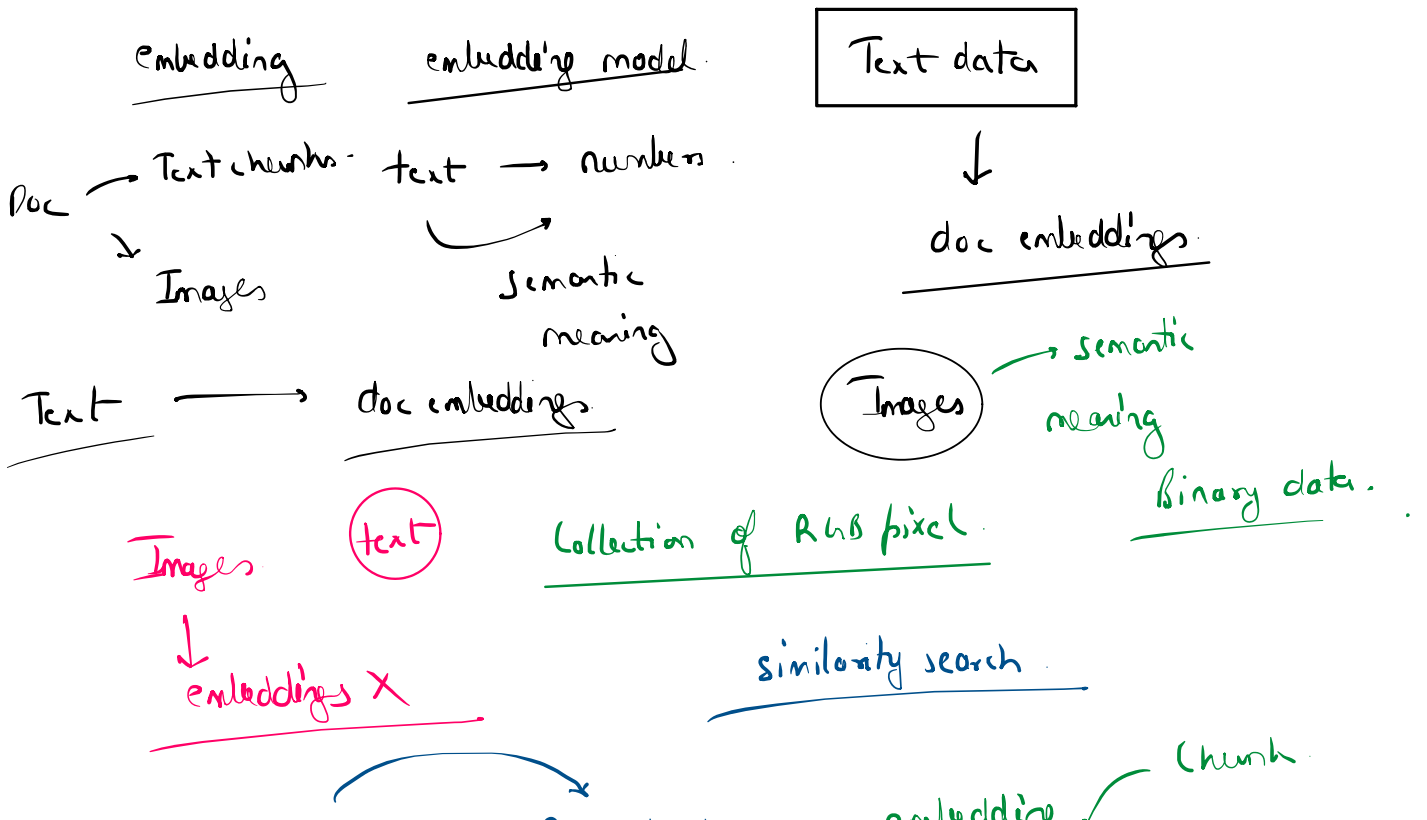
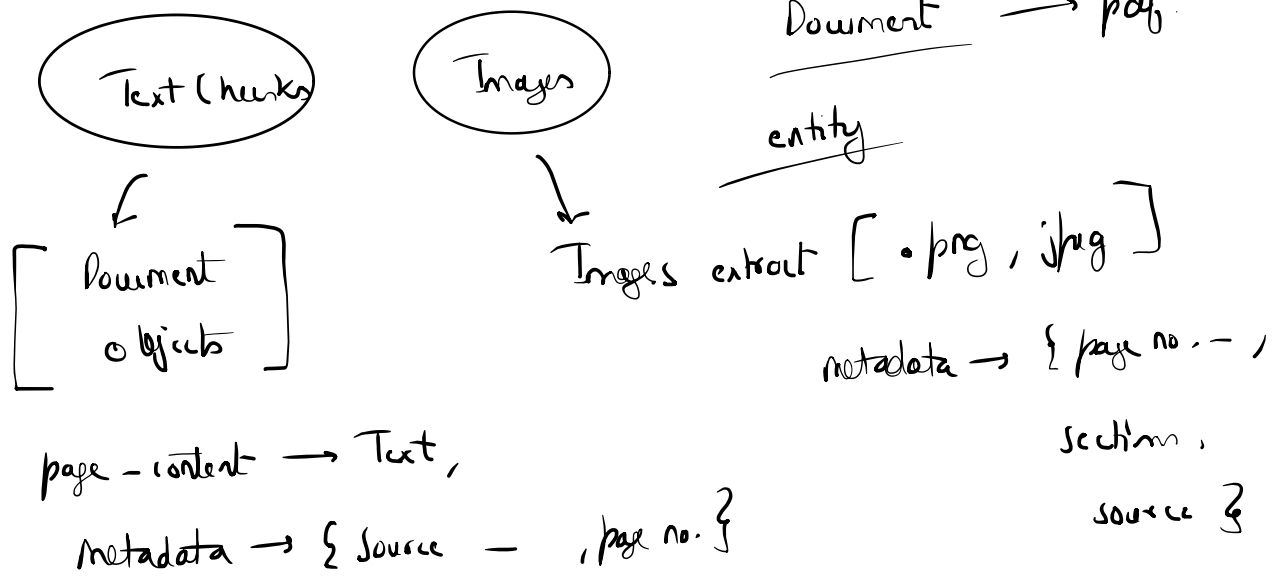
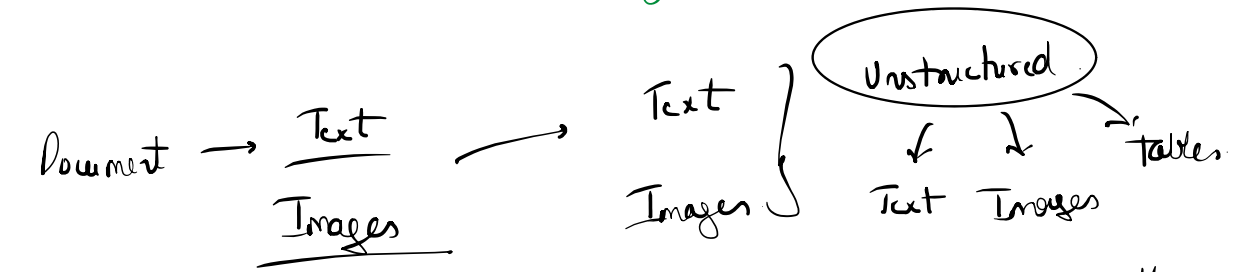
Retrieve ← Top k chunks

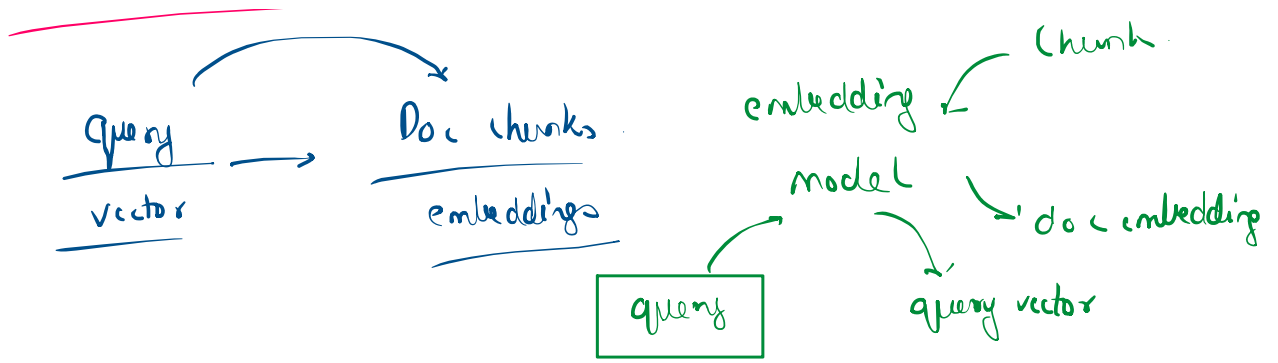




embedding model. → semantic meaning × doc embedding ×

confuse





images → embeddings ~~x~~

similarity search

text → Dense vector (dim)

image → Binary data. (Resolution)

RAH ~~x~~

Similarity

Similarity search

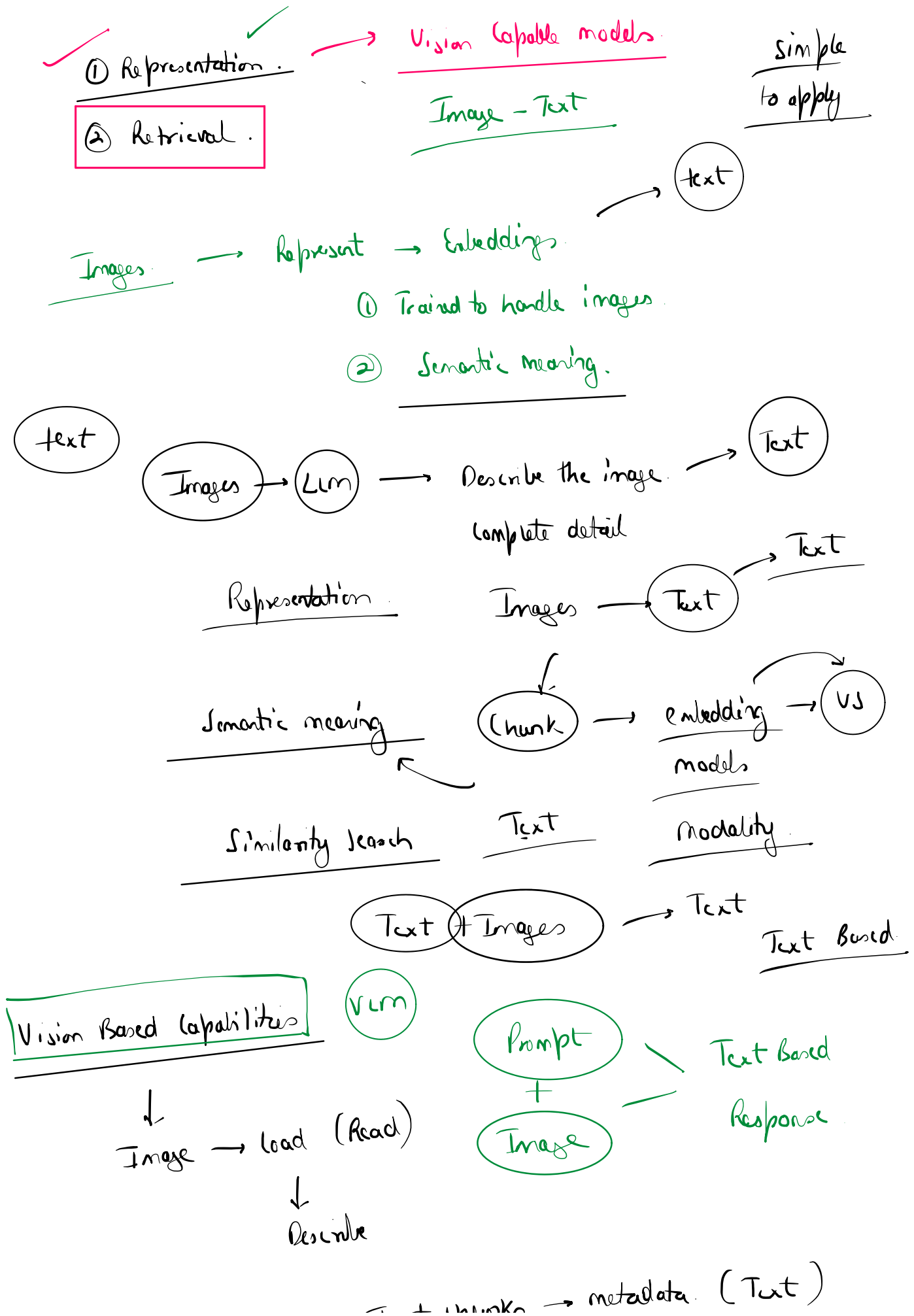
Retrieval

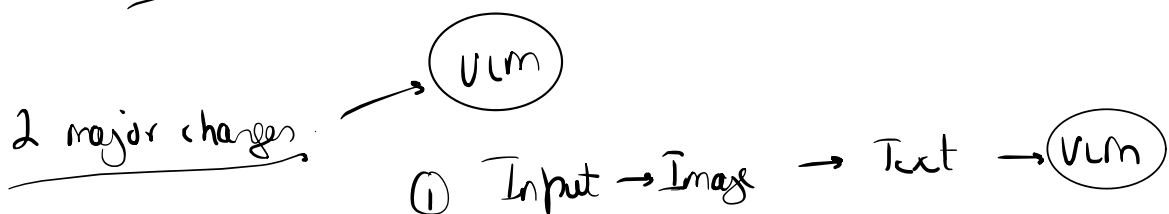
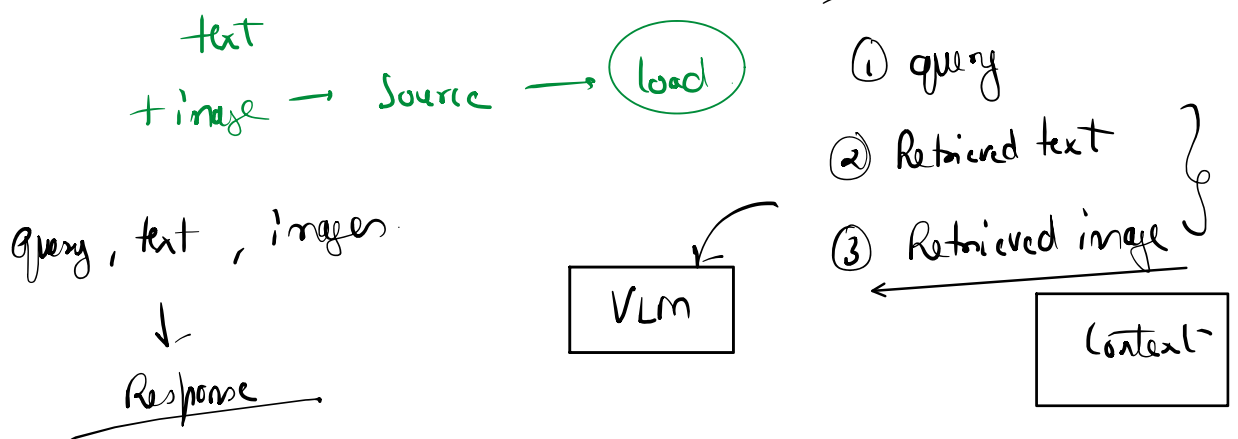
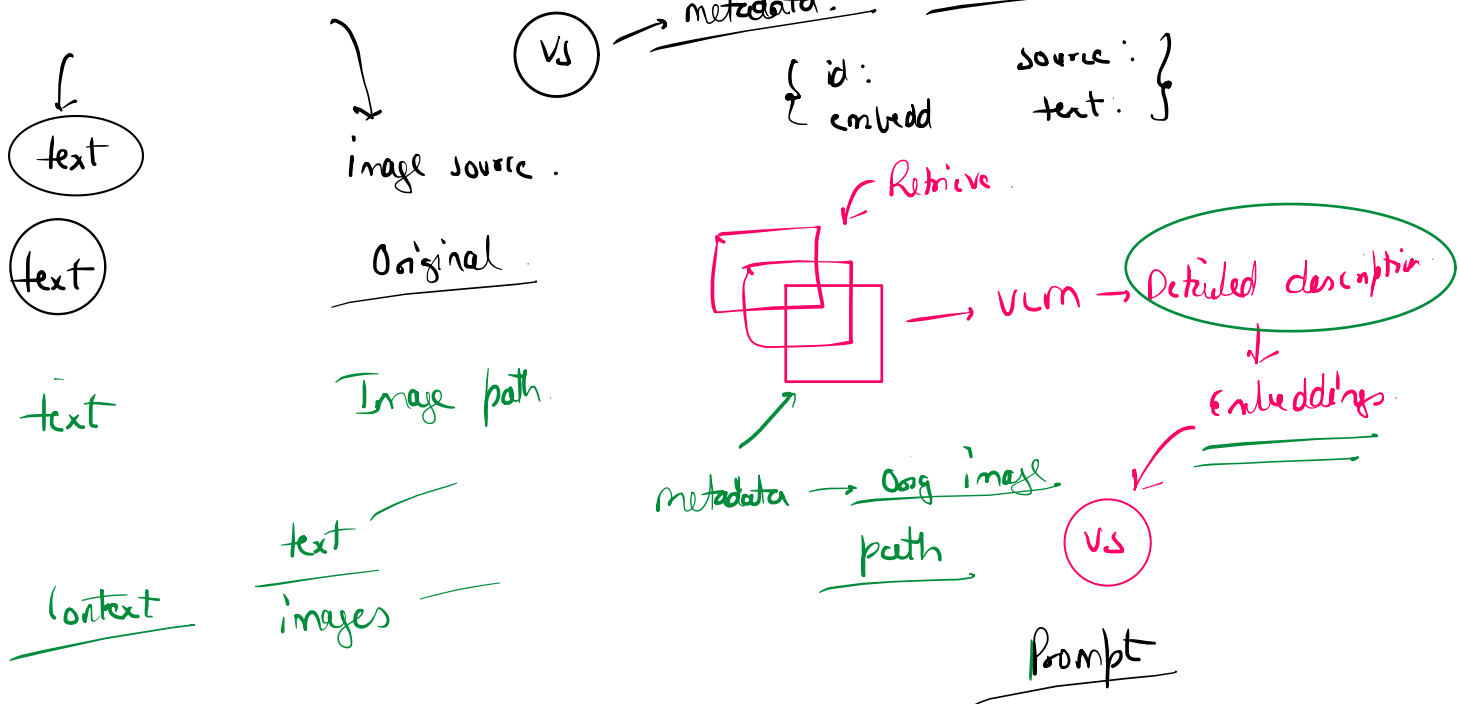
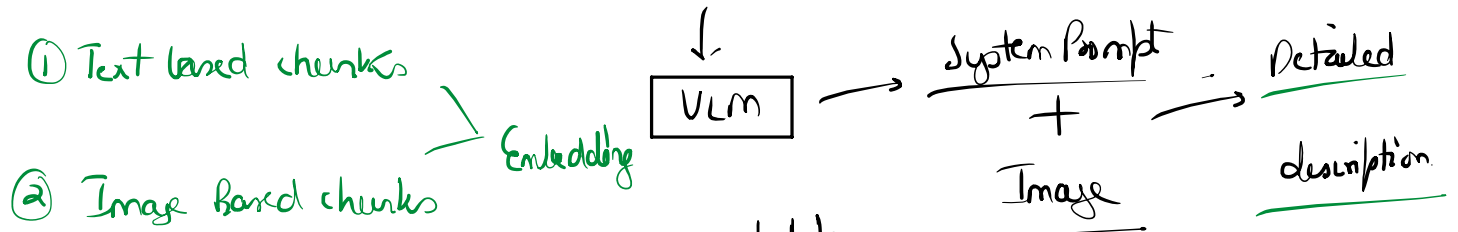
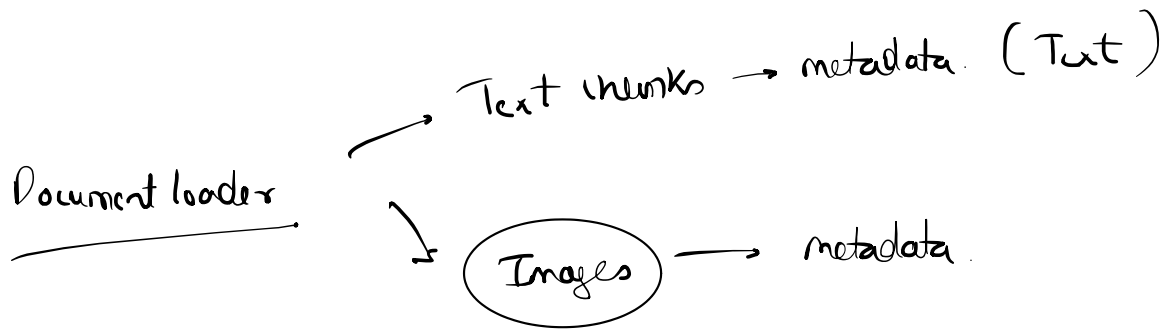
- ✓ ① Representational Problem → Images → embeddings
- ✓ ② Retrieval problem → Similarity search Text → semantic meaning
Comparison x

Images ~~x~~

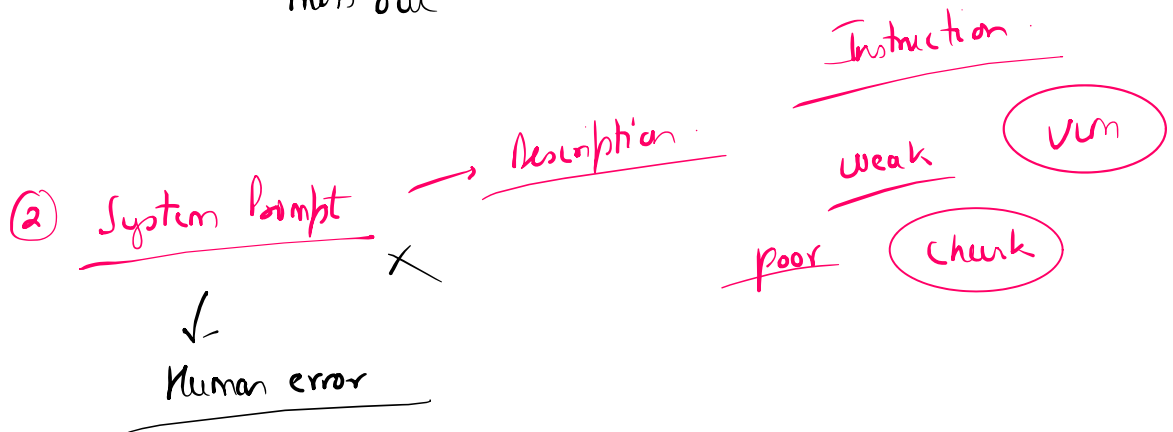
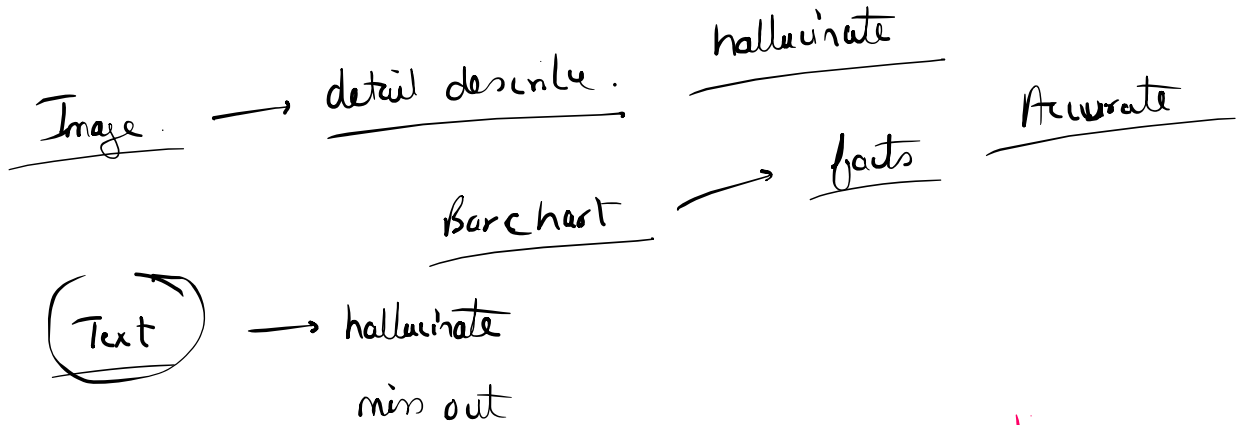
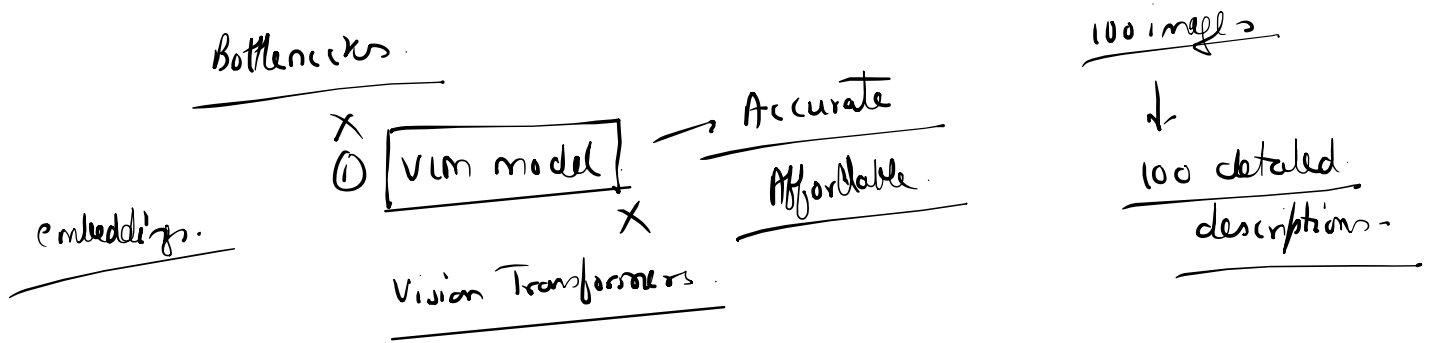
Strategy 1 → More common (less accurate)

Strategy 2 → less common (More Accurate)





② Generation → Text + Image → VLM



Accuracy ↑

Production grade

Multimodal embedding

model

X VLM → Result description of image.

VLM

Images → embedding vectors.

Different model

Text

Image

Document loading

Text

→ embedding model →

embedding vectors

images

→ specialized embedding

model

→ embedding vectors.

Completely different

Architecture

semantic meaning

image properties

embedding vectors

Representation ✓

Retrieval X

semantic meaning ↔ properties

image embedding

dimension

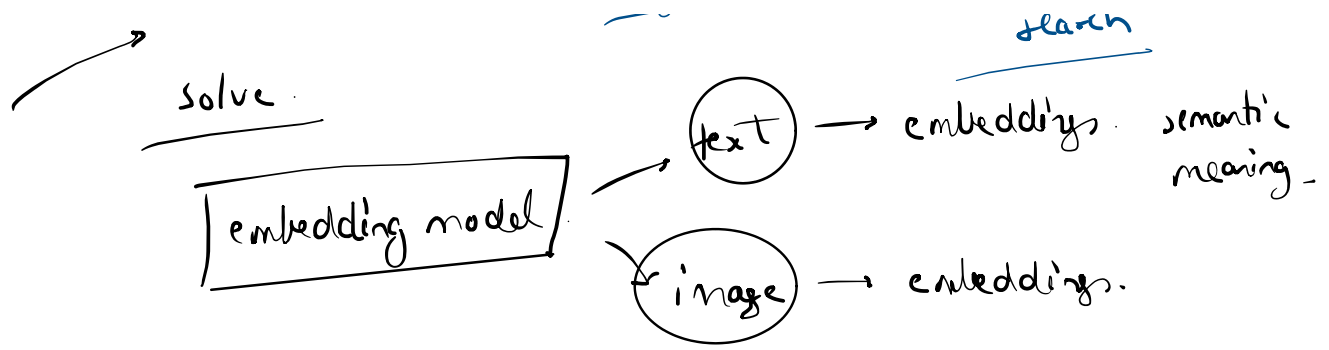
Text embedding

dimension

features

similarity search

→ solve



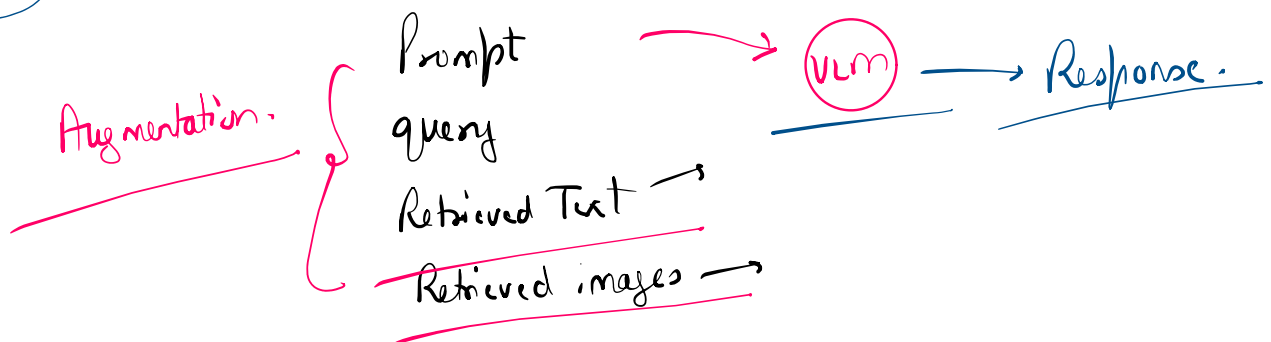
text image Dimensionality same. } similarity search.
features same.

special embedding

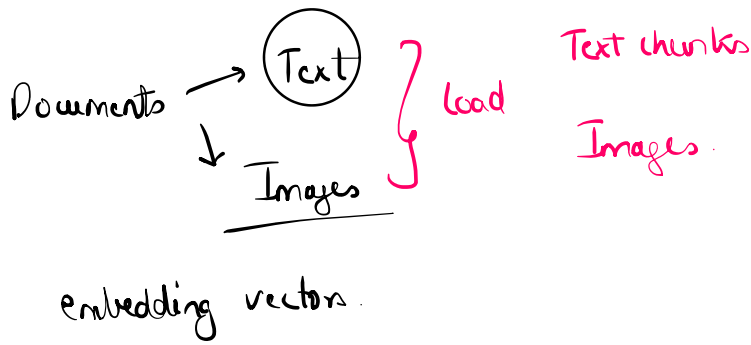
① Representation problem.

② Similarity search → Text, Image → output
comparable

VS



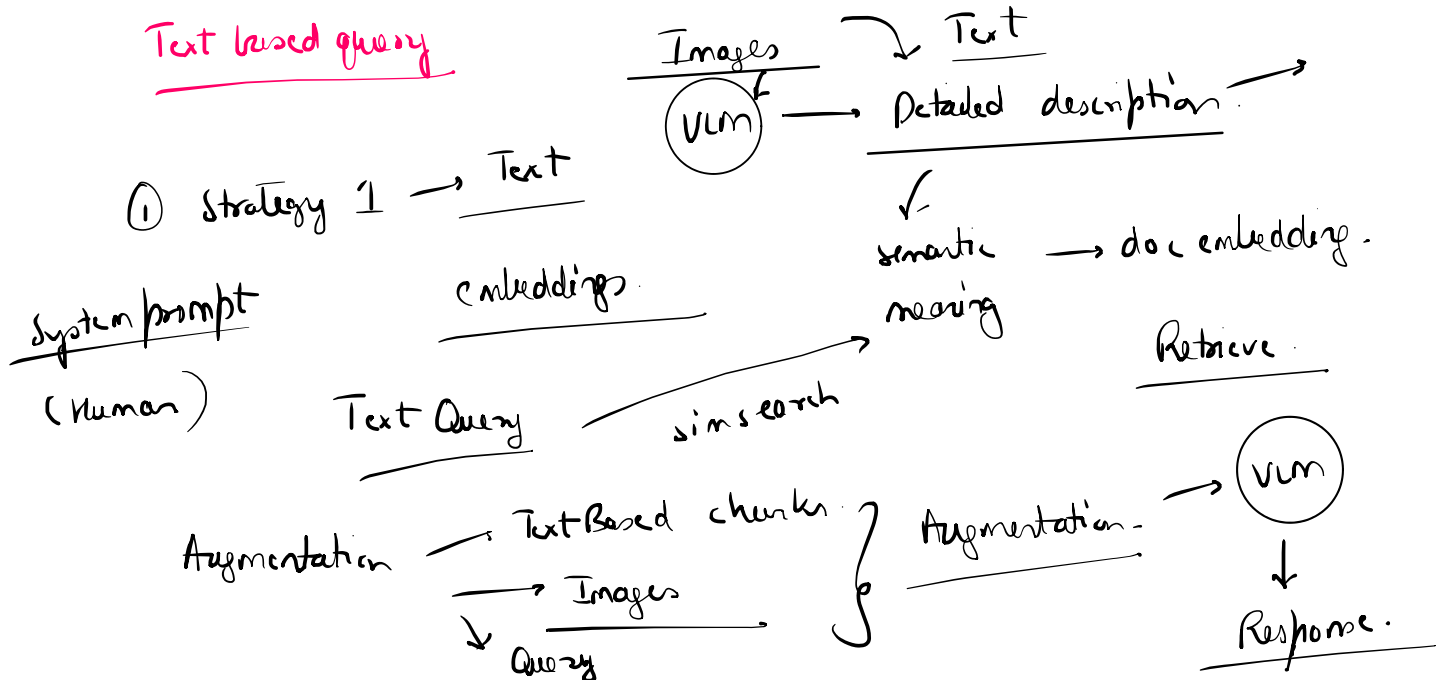
CLIP(Contrastive Language-Image Pre-training)



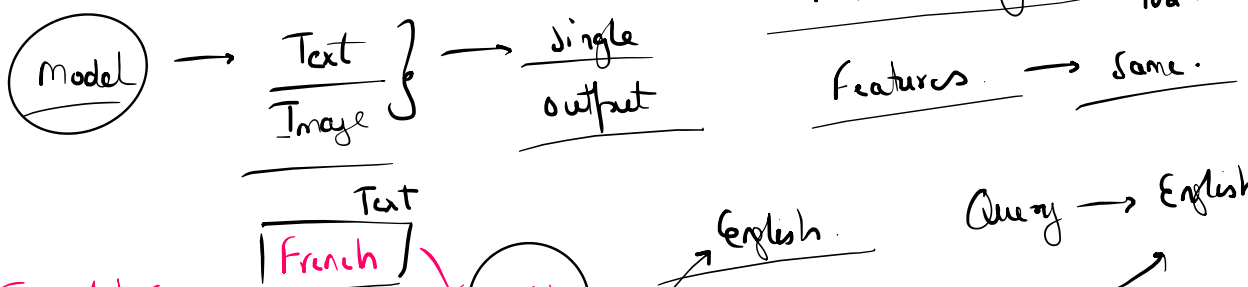
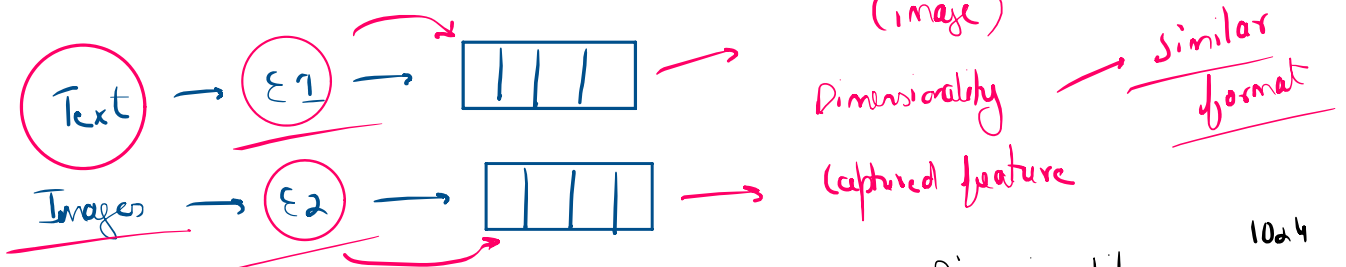
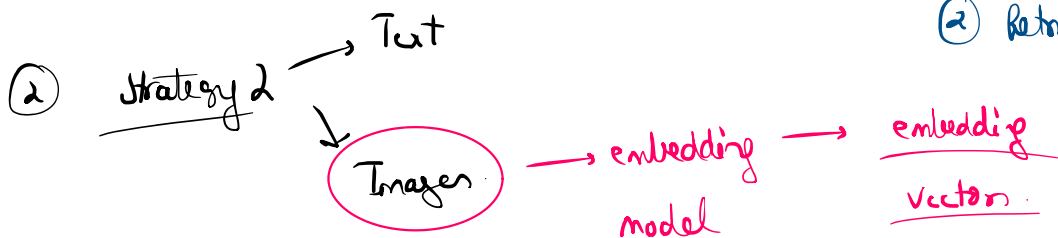
- ① Representational problem
- ② Retrieval problem

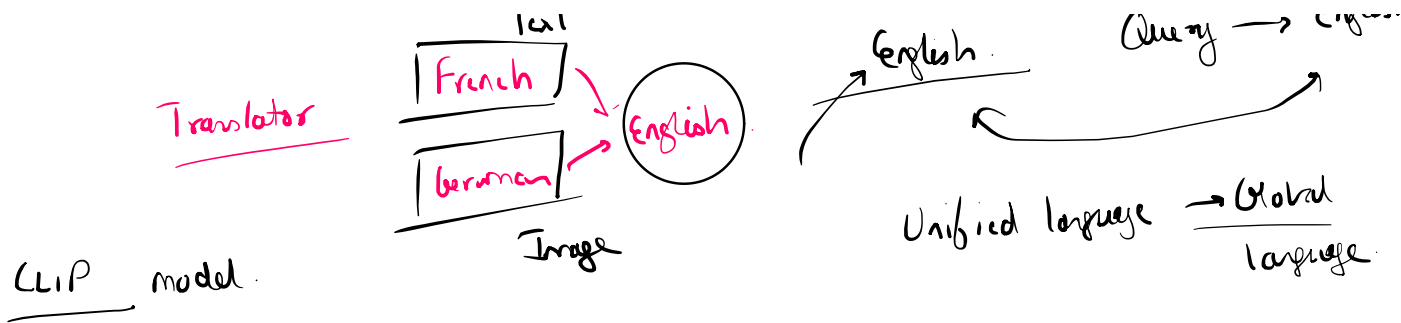
↓ similarity search

Text based query

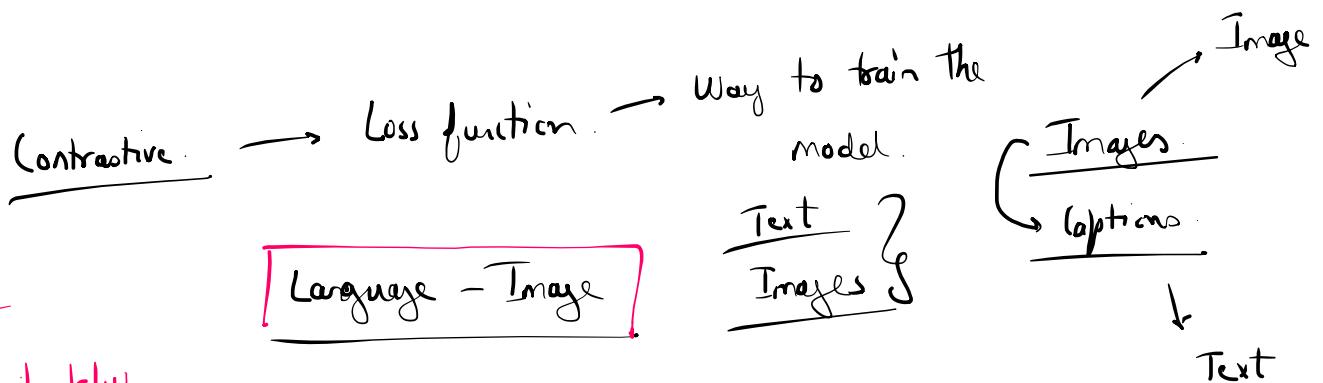
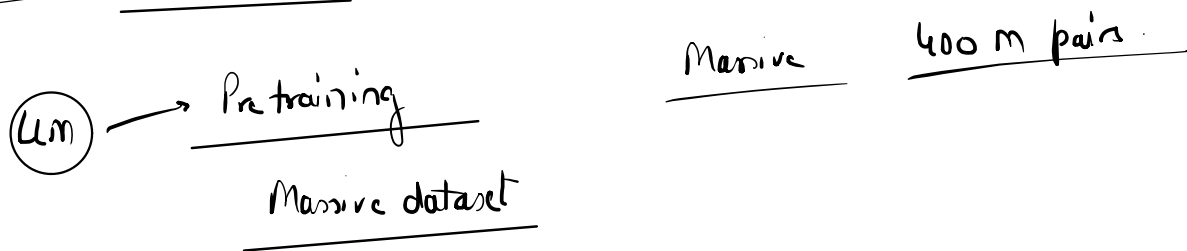


- ① Representation → embeddings
- ② Retrieval → similarity search





Contrastive language-Image Pretraining Model.



Model

Relationship b/w

Image → Text

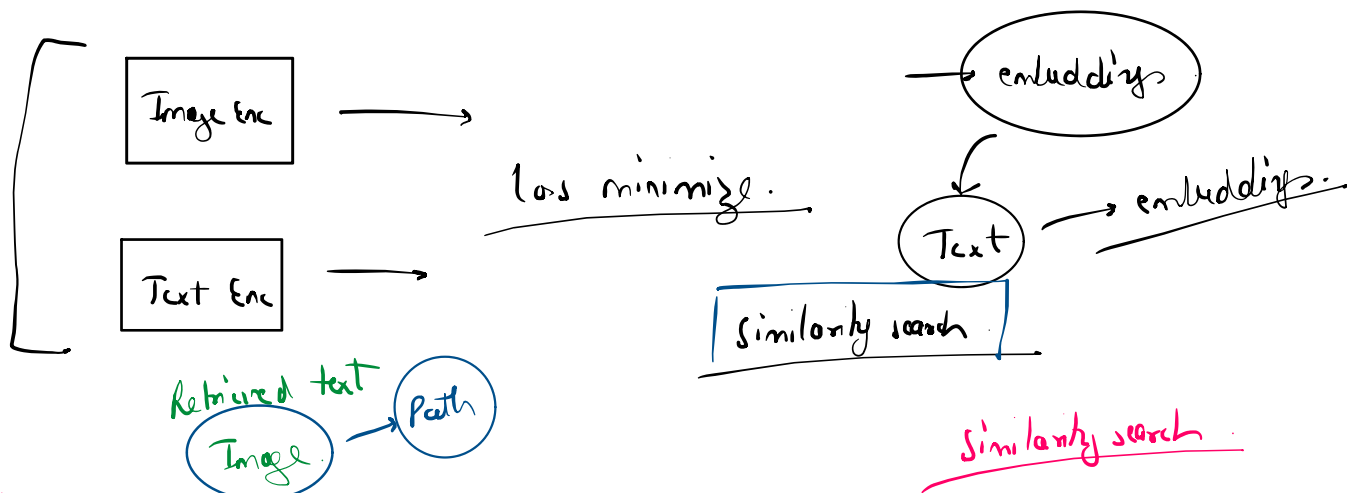
→ Single output

400 m Pairs

Image

Short Description

Image: [photo of a golden retriever playing fetch]
Caption: "a golden retriever playing fetch in a park"



"a dog playing outside"
[golden retriever image]

→ [0.21, -0.54, 0.87, ...]
→ [0.23, -0.51, 0.85, ...] ← very close!

"quarterly revenue chart" → [0.91, 0.12, -0.33, ...] ← far from the dog vectors

At ingestion time:

PDF page with a revenue chart image

→ CLIP image encoder

→ vector stored in your vector DB (with a pointer to the image)

text

image → path

pdf/page

Text
Image

At query time:

User query: "What were the Q3 revenue trends?"

→ CLIP text encoder

→ query vector

→ cosine similarity against all stored vectors

→ retrieves the revenue chart image (because their vectors are close)

→ passes image + query to a vision LLM for the final answer

Text chunk

Similarity Search

Augmentation

Score

text

Image path

① VLM → System Prompt

① Representational step

Image

② Text

Text → E1
image → E2

Grounded response

Unified Model
Images Text → embeddings

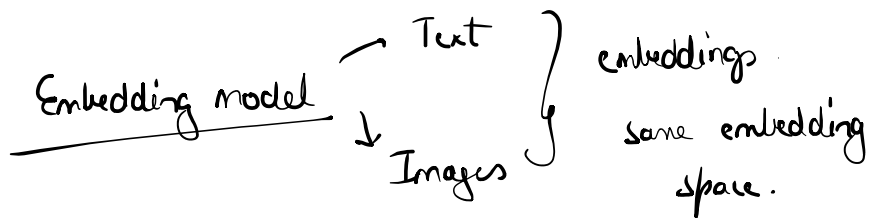
② Retrieval → Query

VLM

Text chunks
Images
Query

↓
Response

Contrastive Learning



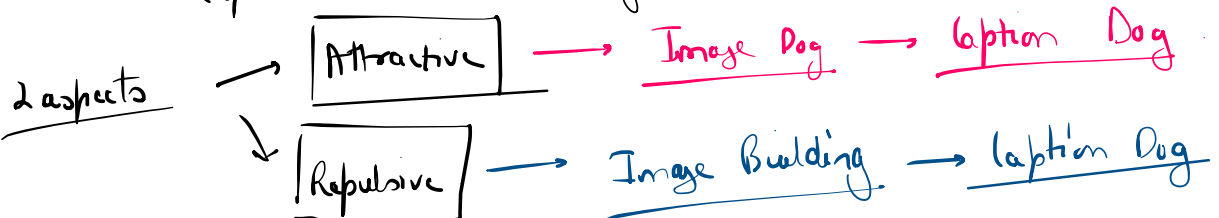
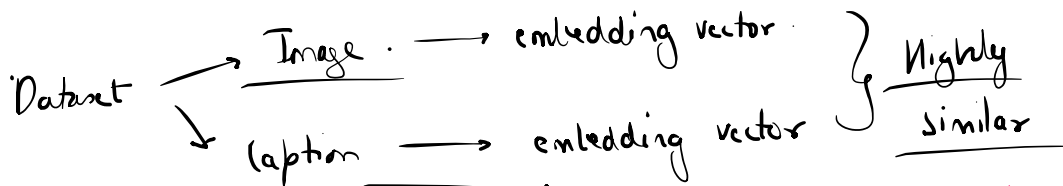
Text
↓ Relationship
Image

Query → Text Based. → Query vector

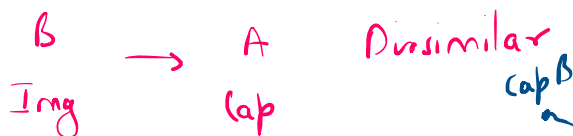
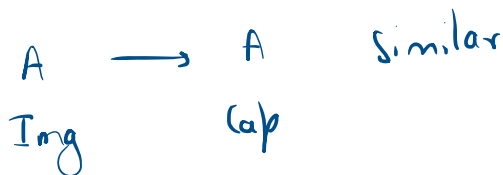
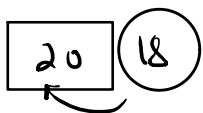
Similarity search

Dog Images
embeddings +
Query.
Retrieve.

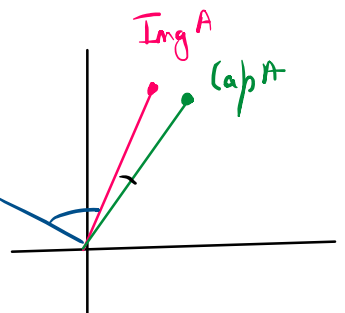
Images + Text → Contrastive learning



Accuracy.



Contrastive learning



Contrastive learning

Image A ↔ Caption A	✓ (should be close)
Image B ↔ Caption B	✓ (should be close)
Image A ↔ Caption B	✗ (should be far apart)
Image B ↔ Caption A	✗ (should be far apart)

u

Image
Caption
Pair

encoders.

Text enc

Image enc

- Pair 1: [Image of a dog] ↔ "a dog playing in the park"
- Pair 2: [Image of a revenue chart] ↔ "Q3 revenue increased by 20%"
- Pair 3: [Image of a mountain] ↔ "a snowy mountain at sunset"
- Pair 4: [Image of a cat] ↔ "a cat sleeping on a couch"

u, images

Image encoder output (raw):

- I1 = [2.1, 0.3, -1.4, 0.8] ← dog image
- I2 = [0.5, 1.9, 0.2, -0.7] ← chart image
- I3 = [-1.2, 0.6, 1.8, 0.4] ← mountain image

← Image vectors

using

Captions

Image encoder output (raw):
 $I1 = [2.1, 0.3, -1.4, 0.8]$ ← dog image
 $I2 = [0.5, 1.9, 0.2, -0.7]$ ← chart image
 $I3 = [-1.2, 0.6, 1.8, 0.4]$ ← mountain image
 $I4 = [1.9, 0.1, -1.3, 0.7]$ ← cat image

Text encoder output (raw):
 $T1 = [1.8, 0.4, -1.2, 0.9]$ ← "a dog playing..."
 $T2 = [0.6, 1.7, 0.3, -0.8]$ ← "Q3 revenue..."
 $T3 = [-1.0, 0.5, 1.9, 0.3]$ ← "a snowy mountain..."
 $T4 = [1.7, 0.2, -1.1, 0.6]$ ← "a cat sleeping..."

← Image Vectors

← Text Vectors

N → Pair

1 Image Vector

Captions

Similarity matrix

Similarity

NxN

Cosine Similarity

4 Caption vectors

	T1 (dog)	T2 (chart)	T3 (mtn)	T4 (cat)
I1 (dog)	[0.98	0.05	0.02	0.81]
I2 (chart)	[0.06	0.97	0.08	0.04]
I3 (mtn)	[0.03	0.07	0.96	0.05]
I4 (cat)	[0.79	0.03	0.04	0.97]

16 sim.

↑
Images

Image A → Cap A
Attractive.
Loss.
Repulsive.

$\left\{ \begin{array}{l} \text{Image A} \rightarrow \text{Cap A} \\ \text{Image B} \rightarrow \text{Cap B} \end{array} \right\}$

Attractive Loss
Image → Cap.

$\left\{ \begin{array}{l} \text{Image A} [0, 0, 0, 0, 0] \\ \text{Cap A} [0, 0, 0, 0, 0] \end{array} \right\} \text{ Similarity } 1$

$\left\{ \begin{array}{l} \text{Image B} [0, 0, 0, 0, 0] \\ \text{Cap B} [0, 0, 0, 0, 0] \end{array} \right\} 1$

Training



Representational collapse

Image embeddings
Caption embeddings

Repulsive.

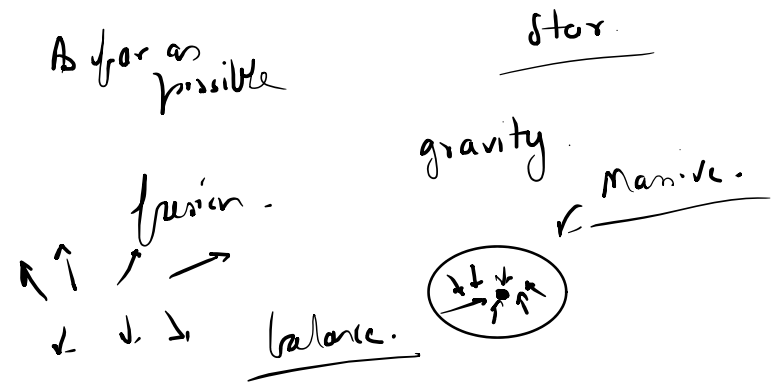
Img A → Cap A } Sim ↑ Attractive.

Representational collapse.

very close to each other

Img A → cap B } sim ↓ Repulsive.
As far as possible

Balancing act



maximise
Attractive

Img A → cap A
Similarity Matrix

	T1 (dog)	T2 (chart)	T3 (mtn)	T4 (cat)
I1 (dog)	0.98	0.05	0.02	0.81
I2 (chart)	0.06	0.97	0.08	0.04
I3 (mtn)	0.03	0.07	0.96	0.05
I4 (cat)	0.79	0.03	0.04	0.97

Loss

Contrastive

Categorical crossentropy → ML } multi class classification.
→ DL

actual class. → score. → 0

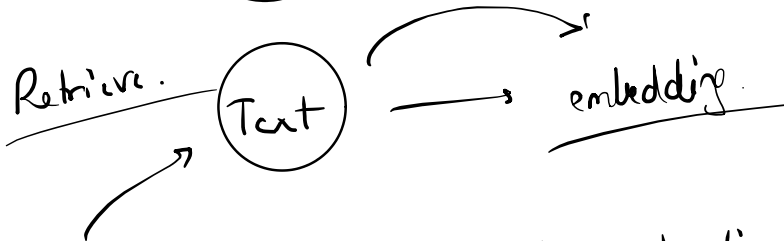
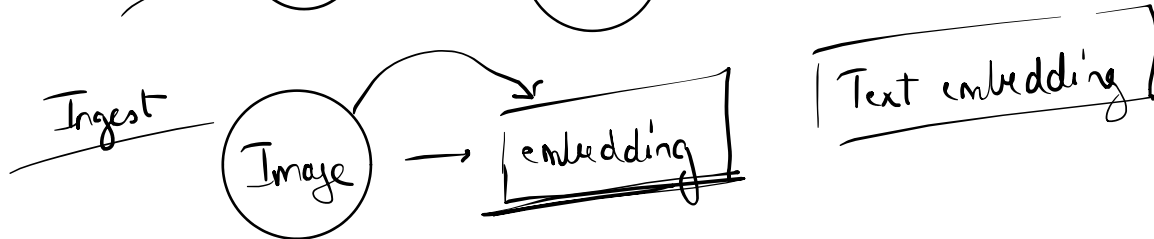
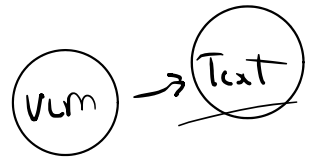
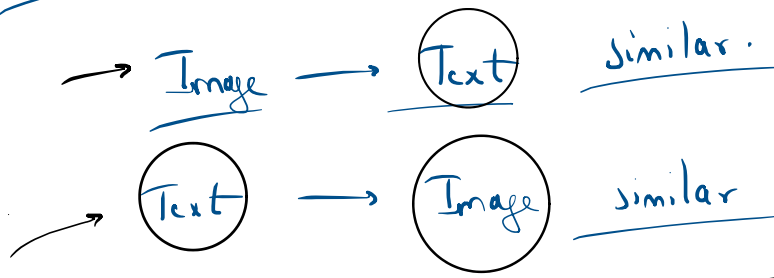
-1
1

Scores in the diagonals should be approaching 1 → Attractive

↑ Repulsive

Scores at non-diagonal places should not loss ↓

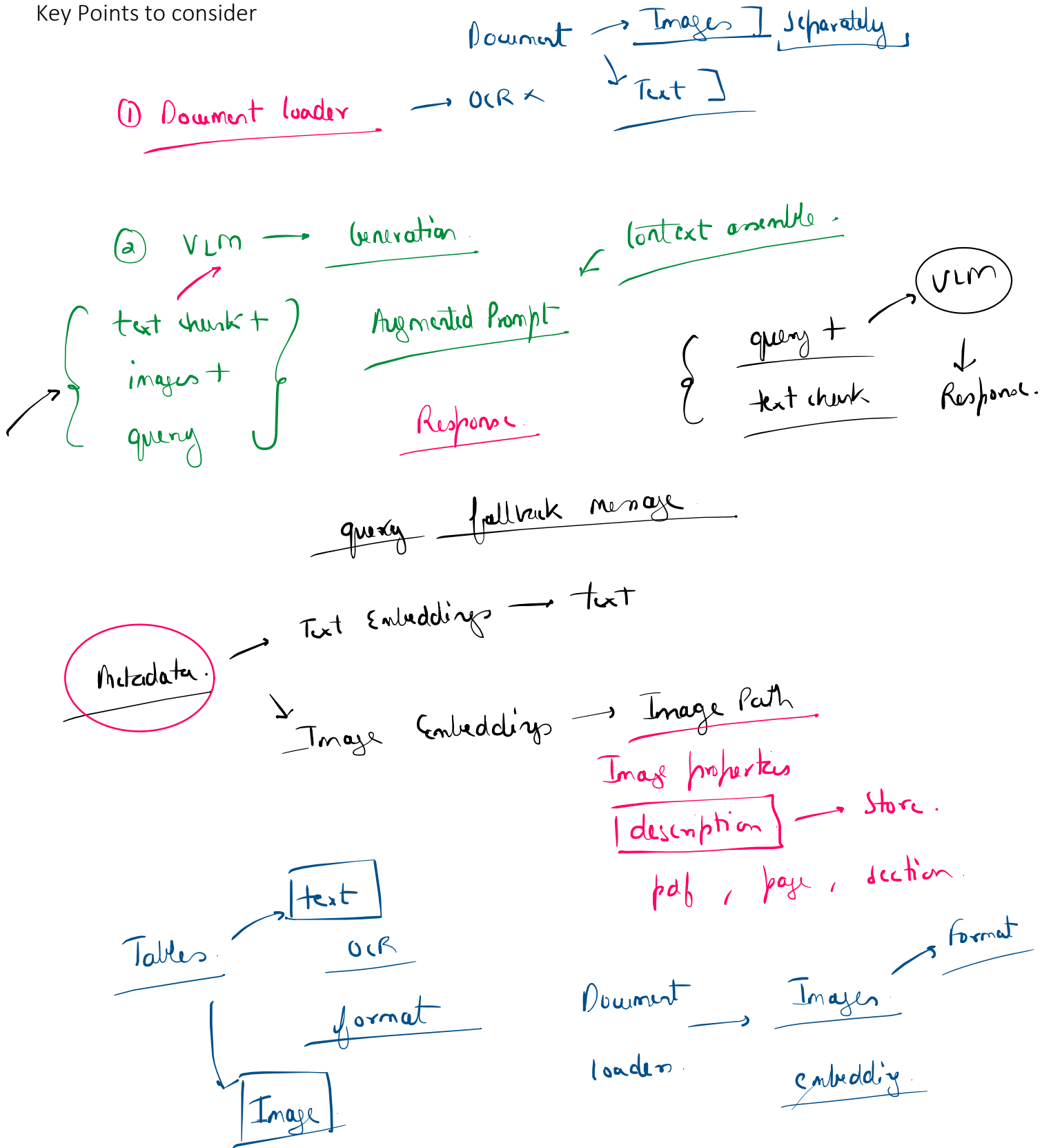
Repulsive. $\downarrow$ $\text{loss} \downarrow$ approach 1



• •
embedding
space

symmetric contrastive learning

Key Points to consider



Code